

RISCV - 8×8 GEMM



차상훈



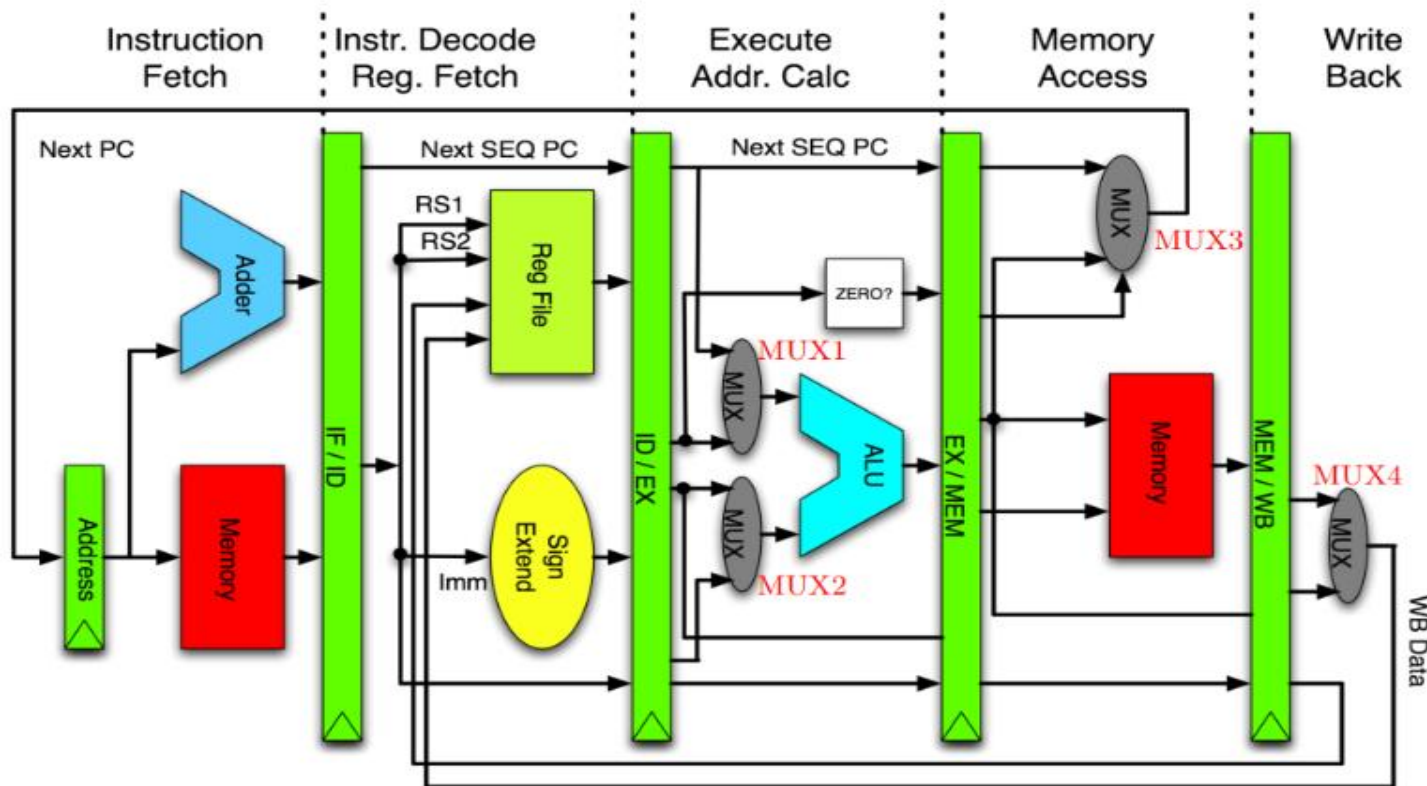
CONTENTS

- Overall Processor Architecture
 - Multicycle RISC-V
- Key Changes compared to the Baseline
 - Branch prediction unit
 - Reduced alu functionality
 - Forwarding unit (deleted)
- Assembly code optimization
 - Shift via Self-Addition
 - Loop unrolling & 4×4 Register Tiling
- PPA analysis & Final PPA result
- Trade-offs & Constraint Compliance Verification



Overall Processor Architecture

5-Stage Pipeline Structure for multi cycle RISC-V32I



- Standard RV32I 5-stage pipeline
 - IF-ID-EX-MEM-WB
- Forwarding unit for Stall mitigation(deleted)
- Branch prediction unit

Key Changes compared to the Baseline

Branch prediction unit

```
module BRANCH_PRED (  
    input          CLK,  
    input          RESET,  
  
    input [31:0] IF_PC,  
    input [31:0] IF_INSTR,  
  
    input          EX_IS_BRANCH,  
    input          EX_PRED_ACTIVE,  
    input [31:0] EX_PC,  
    input          EX_BRANCH_TAKEN,  
  
    output         PRED_TAKEN,  
    output         PRED_ACTIVE  
);
```

- Minimizes pipeline stalls and flushes in frequent GEMM loops
- Eliminates control hazards through dynamic branch prediction

```
// 축소된 테이블  
reg [0:0] LHIST [0:15]; // 16 entry x 1bit  
reg [1:0] PHT [0:31]; // 32 entry x 2bit
```

- LHIST: local history table(7NT 1T, 1bit 최적화)
- PHT: pattern history table



Key Changes compared to the Baseline

Reduced alu functionality

```
module alu (  
    input [31:0] a,  
    input [31:0] b,  
    input [ 1:0] alucontrol,  
    output reg [31:0] result,  
    output zero  
);  
always @(*) begin  
    case (alucontrol)  
        2'b00: result = a + b; // ADD  
        2'b01: result = a - b; // SUB  
        2'b10: result = a & b; // AND  
        default: result = 32'd0;  
    endcase  
end  
  
assign zero = (result == 32'd0);  
  
endmodule
```

- Maximizes area efficiency by restricting functional states to 3 essential operations(add, sub for beq, and) for GEMM computing



Key Changes compared to the Baseline Forwarding unit(**deleted**)

```
module FORWARD (  
    input  [4:0] ID_EX_RS1,  
    input  [4:0] ID_EX_RS2,  
  
    input  [4:0] EX_MEM_RD,  
    input      EX_MEM_REGWRITE,  
  
    output      FORWARD_A,  
    output      FORWARD_B  
);  
    assign FORWARD_A = EX_MEM_REGWRITE && (EX_MEM_RD != 5'd0) && (EX_MEM_RD == ID_EX_RS1);  
    assign FORWARD_B = EX_MEM_REGWRITE && (EX_MEM_RD != 5'd0) && (EX_MEM_RD == ID_EX_RS2);  
  
endmodule
```

- **Critical path**->instruction scheduling



Assembly code optimization

Shift via Self-Addition

```
    add x5, x5, x6      # product += A*1
bit0_skip:
    # — bit 1 : B & 2 (shifted_A = A*2) —
    andi x7, x30, 2     # x7 = val_B & 2
    add x6, x6, x6      # shifted_A = A*2 (항상 실행, 분기 앞)
    beq x7, x0, bit1_skip # x7: andi→beq gap=1 ✓
    add x5, x5, x6      # x6: add→add gap=1 ✓
bit1_skip:
    # — bit 2 : B & 4 (shifted_A = A*4) —
    andi x7, x30, 4
    add x6, x6, x6      # shifted_A = A*4
    beq x7, x0, bit2_skip
    add x5, x5, x6
bit2_skip:
    # — bit 3 : B & 8 —
    andi x7, x30, 8
    add x6, x6, x6      # shifted_A = A*8
    beq x7, x0, bit3_skip
    add x5, x5, x6
bit3_skip:
```

- Shift A by self-addition (add A, A, A)
- Bit-masking on B data
- Accumulate to C if the result is 1
- Cycles: Approx. 13,000
- IMEM: 70

Assembly code optimization

Loop unrolling & 4x4 Register Tiling

```
row_block:
    # Clear 16 accumulators for the first 4x4 tile.
    addi x16, x0, 0
    addi x17, x0, 0
    addi x18, x0, 0
    addi x19, x0, 0
    addi x20, x0, 0
    addi x21, x0, 0
    addi x22, x0, 0
    addi x23, x0, 0
    addi x24, x0, 0
    addi x25, x0, 0
    addi x26, x0, 0
    addi x27, x0, 0
    addi x28, x0, 0
    addi x29, x0, 0
    addi x30, x0, 0
    addi x31, x0, 0
    addi x5, x0, 256
    addi x3, x0, 256
    addi x2, x0, 1
```

```
t0_k:
    # Load A[row+0..3][k] and B[k][col+0..3].
    lw x8, 0(x4)
    lw x9, 32(x4)
    lw x10, 64(x4)
    lw x11, 96(x4)
    lw x12, 0(x5)
    lw x13, 4(x5)
    lw x14, 8(x5)
    lw x15, 12(x5)
```

```
t0_bit:
    # Decompute bit tests for four B columns
```

- Load 4 rows of A and 4 columns of B per k loop to accumulate 16 elements at once
- Pointer-based loop termination (no counter register needed)
- Unconditional branch via beq x0, x0 (replaces jal to reduce instruction count)
- Cycle: 3600~3800
- IMEM: 172



- PPA analysis

Only Shift via Self-Addition

CLOCK period	3ns
Power	3.36mW
Area	31186
Average Cycle	12856
Execution Time	38us
Energy	129nJ

**Loop unrolling &
4×4 Register Tiling**

CLOCK period	3ns
Power	3.63mW
Area	35110
Average Cycle	3704
Execution Time	11.1us
Energy	40.5nJ



- PPA analysis

- **Loop unrolling & 4×4 Register Tiling**
- **Creates a critical path bottleneck due to all add instructions routing through the forwarding MUX**

- **if $CLK \leq 3n$, slack violation**

- **Instruction scheduling -> We can delete forwarding unit**

```
addi x5, x0, 256
t0_bit:
    and x7, x12, x2
    beq x7, x0, t0_c0_skip
    add x16, x16, x8
    add x20, x20, x9
    add x24, x24, x10
    add x28, x28, x11
t0_c0_skip:
    and x7, x13, x2
    beq x7, x0, t0_c1_skip
    add x17, x17, x8
    add x21, x21, x9
    add x25, x25, x10
    add x29, x29, x11
```



```
lw x15, 12(x5)
t0_bit:
    # Precompute bit tests for four B columns.
    # x2 is the current bit mask.
    and x7, x12, x2
    and x1, x13, x2
    and x6, x14, x2
    # If B[k][col0] has this bit, add shifted A to col0 accumulators.
    beq x7, x0, t0_c0_skip
    add x16, x16, x8
    add x20, x20, x9
    add x24, x24, x10
    add x28, x28, x11
t0_c0_skip:
    # If B[k][col1] has this bit, add shifted A to col1 accumulators.
    and x7, x15, x2
    beq x1, x0, t0_c1_skip
    add x17, x17, x8
    add x21, x21, x9
    add x25, x25, x10
    add x29, x29, x11
```

- PPA analysis

**Loop unrolling &
4×4 Register Tiling**

CLOCK period(max)	3ns
Power	3.63mW
Area	35110
Average Cycle	3704
Execution Time	11.1us
Energy	40.5nJ

**Loop unrolling & 4×4
Register Tiling
with no forwarding unit**

CLOCK period(max)	2.55nsec
Power	4.16mW
Area	34272.76
Average Cycle	3793
Execution Time	9.67usec
Energy	40.3nJ

• PPA analysis

- Additional improvement in Branch prediction unit
- Excluded random B-matrix bit-check branches from the predictor and reduced history bits from 2 to 1, cutting branch predictor storage by 50% (160 → 80 bits) with no accuracy loss.
- B masking beq는 random-> 예측 대상 제외
- Bit loop, k loop beq는 항상 7NT 1T
- 00(NT→NT), 01(NT→T) 두 가지만 실제로 나타남. 10(T→NT), 11(T→T) 조합은 절대 발생 X (T 다음엔 항상 다시 NT가 시작: PHT table에서 LHIST가 1일 때 00고정).

```
reg [1:0] LHIST [0:15]; // 16 entry × 2bit
reg [1:0] PHT [0:63]; // 64 entry × 2bit
```



```
reg [0:0] LHIST [0:15]; // 16 entry × 1bit
reg [1:0] PHT [0:31]; // 32 entry × 2bit
```

Power	area	energy
4.16mW	34272.76	40.3nJ



Power	area	energy
3.92mW	32291.44	37.9nJ



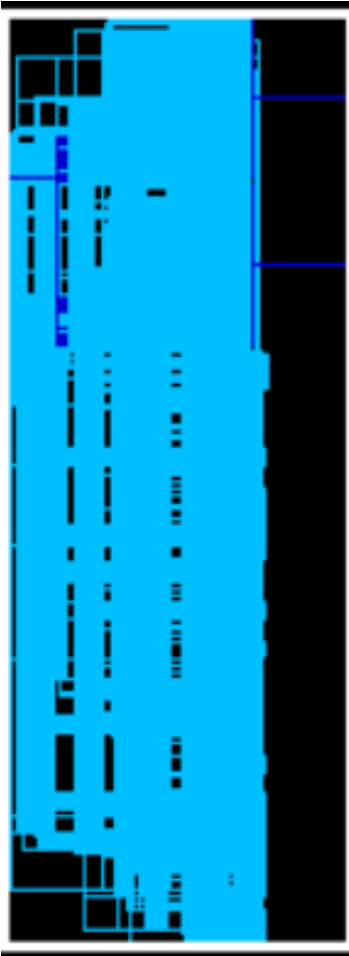
- Final PPA result

**Loop unrolling & 4×4
Register Tiling
with no forwarding unit
& Updated Bran_Pred**

CLOCK period(max)	2.4nsec
Power	4.1255mW
Area	32421.42
Average Cycle	3793
Execution Time	9.10usec
Energy	37.6nJ



Trade-offs



- **Pipelining vs. Cycle Count**
 - Deeper pipelining enables higher clock frequencies but increases the overall cycle count
- **Loop Unrolling vs. Area**
 - Loop unrolling reduces cycles but increases instruction memory (IMEM) requirements, leading to larger area overhead
- **Frequency vs. Power**
 - Higher clock frequencies directly elevate dynamic power consumption.
- **Timing Optimization vs. Gate Count**
 - Achieving tight timing margins at high frequencies forces synthesis tools to insert large buffers and inverter trees, expanding the area.

DC synthesis schematic



Constraint Compliance Verification

```
#####  
## CLOCK  
## uncertainty, transition, input/output delay clock period의 40%, load cap 기본값100  
#####  
create_clock -name CLOCK -period 2.4 [get_ports CLK]  
set_clock_latency 1 [get_clocks CLOCK]  
set_clock_uncertainty -setup 0.5 [get_clocks CLOCK]  
set_clock_uncertainty -hold 0.1 [get_clocks CLOCK]  
set_clock_transition 0.5 [get_clocks CLOCK]  
#####  
## RESET  
#####  
set_false_path -from [get_ports RESET] -to [all_registers]  
  
#####  
## INPUTS (CLOCK PERIOD 40%)  
#####  
set_input_delay 0.96 -clock CLOCK [remove_from_collection [all_inputs] [get_ports {CLK RESET}]]  
set_input_transition 0.1 [remove_from_collection [all_inputs] [get_ports {CLK RESET}]]  
  
#####  
## OUTPUTS (CLOCK PERIOD 40%)  
#####  
set_output_delay 0.96 -clock CLOCK [all_outputs]  
set_load 100 [all_outputs]
```

